MODULE 22 · DOMAIN 3 - CLOUD NATIVE APPLICATION DELIVERY

Delivery fundamentals

CI and CD, build once and promote, and the strategies for releasing a new version.

DOMAIN 3 of 4 EXAM WEIGHT 16% MODULE 22 of 35

admin@sthithapragna.com

Delivery is **the path from a commit to production**. The goal is to make it frequent, automatic and boring.

admin@sthithapragna.com

Continuous integration versus continuous delivery

CI

Integrate and verify

- Merge small changes often
- Every change is built automatically
- Tests and scans run on every change
- Output: a tested artefact, such as an image

CD

Get it into environments

- Takes the artefact from CI
- Deploys to staging, then production
- Promotes the same artefact
- Output: a running release

THE TIEBREAKER

CI produces a trustworthy artefact. CD moves that artefact safely into environments.

Continuous delivery versus continuous deployment

CONTINUOUS DELIVERY

Always releasable

- Every change could go to production
- A person approves the final step
- Common where releases need sign-off
- Release is a business decision

CONTINUOUS DEPLOYMENT

Always released

- Every passing change goes to production
- No human approval step
- Needs strong automated tests
- Release is automatic

THE TIEBREAKER

The only difference is the human gate before production. Same initials, CD, for both.

A typical pipeline

01
Commit
A developer merges a small change.

02
Build
CI builds the image, once.

03
Test and scan
Unit tests, vulnerability scan, SBOM, signature.

04
Publish
Push to the registry, tagged with a version.

05
Deploy
Staging first, then promote the same image to production.

ON THE EXAM

Supply chain security from module 19 lives in the test-and-scan stage.

Build once, promote everywhere

- **One image, built once,** runs in every environment. Identified by version tag or digest.
- **Only configuration differs** between environments: ConfigMaps, Secrets, overlays.
- **Never rebuild for production.** A rebuild can pull different dependencies.
- **Never deploy latest.** Module 1 showed why: tags move.

> ON THE EXAM
>
> What you tested must be exactly what you ship. Promote the artefact; do not recreate it.

CI and CD tools you should recognise

TOOL	KNOWN FOR
Jenkins	Long-standing automation server with a huge plugin ecosystem
GitHub Actions, GitLab CI	Pipelines built into the code-hosting platform
Tekton	Kubernetes-native pipelines, defined as custom resources
Argo Workflows	Kubernetes-native workflow engine; part of the graduated Argo project
Argo CD, Flux	GitOps continuous delivery, in module 24

ON THE EXAM

Kubernetes-native tools run each pipeline step as a Pod, and are defined as Kubernetes objects.

Infrastructure as code

- **Infrastructure declared in files,** kept in version control, reviewed like code.
- **Repeatable:** the same files build the same environment every time.
- **Drift is visible:** compare what exists with what the files say.
- **Tools:** Terraform and OpenTofu, Pulumi, and Crossplane, a CNCF project managing cloud resources as Kubernetes objects.

> ON THE EXAM
>
> Kubernetes manifests are infrastructure as code for workloads. The same discipline applies.

Push versus pull deployment

PUSH

The pipeline deploys

- CI runs kubectl or helm against the cluster
- CI holds cluster credentials
- Cluster changes only when a pipeline runs
- Drift goes unnoticed

PULL

The cluster fetches

- An agent in the cluster watches Git
- No cluster credentials outside it
- Changes applied continuously
- Drift is detected and corrected

THE TIEBREAKER

Pull-based delivery is GitOps. Module 24 covers it in depth.

A new version can reach users all at once, gradually, side by side, or to a few first.

Release strategies at a glance

STRATEGY	HOW	TRADE-OFF
Recreate	Stop the old version, then start the new	Simple; causes downtime
Rolling update	Replace Pods gradually	The Deployment default; brief version mix
Blue-green	Run the new version alongside; switch all traffic at once	Instant rollback; double capacity
Canary	Send a small share of users to the new version first	Small blast radius; needs good metrics
A/B testing	Route by user attribute to compare variants	A business experiment, not just a release
Shadow	Copy live traffic to the new version; discard its replies	Real load, no user impact

ON THE EXAM

Deployments give you Recreate and rolling update. The others need extra Services, routing or tools.

Blue-green, step by step

01

Blue is live

All users on version 1, the blue environment.

02

Deploy green

Version 2 runs alongside, taking no traffic.

03

Test green

Smoke tests against green directly.

04

Switch

Point the Service at green. All users move at once.

05

Keep blue

Switch back instantly if needed; retire it later.

ON THE EXAM

The switch is one change, so the rollback is one change too.

Blue-green with plain Kubernetes

Two Deployments, web-blue and web-green, label their Pods track: blue and track: green.

```
apiVersion: v1
kind: Service
metadata: {name: web, namespace: quayside}
spec:
  selector: {app: web, track: green}
  ports: [{port: 80, targetPort: 8080}]

# rollback: set track back to blue
```

5 The switch: one label in the selector moves all traffic.

THE TAKEAWAY

Instant switch and instant rollback, at the cost of running both versions in full.

Canary, step by step

01

5%

A few users get version 2.

02

Watch

Compare errors and latency with version 1.

03

25%

Healthy? Send more.

04

50%

Still healthy? More again.

05

100%

Done. Any bad signal: back to 0%.

ON THE EXAM

A canary limits the damage of a bad release to the few users who saw it.

Canary by replica count versus by traffic weight

REPLICA RATIO

Coarse

- 1 canary Pod among 10 is about 10%
- Share tied to the number of Pods
- Hard to do 1%: you need 100 Pods
- Works with a plain Service

TRAFFIC WEIGHTS

Precise

- Gateway API or mesh weights: exactly 5%
- Share independent of Pod count
- Can route by header, e.g. staff only
- Module 15's HTTPRoute, 90 to 10

THE TIEBREAKER

For precise canaries, split traffic by weight at the Gateway or mesh, not by counting Pods.

Blue-green versus canary

BLUE-GREEN

All at once, instantly reversible

- Every user moves at the same moment
- Rollback is one switch
- Needs a full second environment
- No gradual real-user signal

CANARY

A few first, then more

- Real users exposed gradually
- Bad releases hit few users
- Needs metrics and traffic splitting
- Takes longer to complete

THE TIEBREAKER

'Switch everyone at once, instant rollback': blue-green. 'Expose a few users first': canary.

Progressive delivery: automating the canary

- **Tools step the canary for you,** checking metrics at each stage.
- **Argo Rollouts** replaces a Deployment with a Rollout resource that supports canary and blue-green.
- **Flagger**, from the Flux family, automates canaries with a mesh or gateway.
- **Metrics decide:** error rate or latency above a threshold triggers an automatic rollback.

> ON THE EXAM
>
> 'Canary with automatic rollback on bad metrics': progressive delivery, e.g. Argo Rollouts or Flagger.

Feature flags: deploy is not release

- **Ship code switched off,** then turn features on without a new deployment.
- **Target users:** staff first, then a region, then everyone. Turn off instantly if needed.
- **Separates deployment** (code running) from **release** (users seeing it).
- **OpenFeature**, a CNCF project, is a vendor-neutral API for feature flags.

> ON THE EXAM
>
> Deployment is a technical event; release is a business decision. Flags let them happen at different times.

> At Quayside, every merge to the main branch is built, tested, scanned and then deployed to production automatically.
>
> No person approves any step between the merge and production.

Which practice does this describe?

A Continuous integration, and nothing more

B Continuous deployment

C Continuous delivery with a release gate

D Infrastructure as code

◆ PAUSE AND DECIDE

Which practice does this describe?

A Continuous integration, and nothing more

CI stops at a tested artefact; this goes on to production.

B Continuous deployment

Every passing change reaches production with no human gate.

C Continuous delivery with a release gate

That has a person approving production. This has none.

D Infrastructure as code

About declaring infrastructure, not the release path.

THE SIGNAL

No human approval before production: continuous deployment.

> For the new booking API, Quayside wants every user moved to the new version at the same moment, and an instant way back if anything goes wrong.
>
> It can afford double capacity for an hour during the switch.

Which strategy fits?

A. A rolling update with default settings

B. A canary, starting at 5% of users

C. Recreate: stop old, then start new

D. Blue-green, switching the Service at once

◆ PAUSE AND DECIDE

Which strategy fits?

A A rolling update with default settings

Users are on mixed versions during the rollout.

B A canary, starting at 5% of users

Deliberately moves users gradually, not at once.

C Recreate: stop old, then start new

All at once, but with downtime and a slow way back.

D Blue-green, switching the Service at once

One switch forward, one switch back, double capacity.

THE SIGNAL

'Everyone at once, instant rollback, double capacity acceptable': blue-green.

> Quayside's new fare engine is risky. The team wants 5% of real users on it first, then gradually more.
>
> If its error rate rises above the current version's, it must roll back automatically.

What should they use?

A. A canary, automated by Argo Rollouts or similar

B. Blue-green with a manual switch after testing

C. The Recreate strategy on a Deployment

D. Shadow traffic sent to the new fare engine

◆ PAUSE AND DECIDE

What should they use?

A A canary, automated by Argo Rollouts or similar
Gradual real-user exposure with metric-driven rollback.

B Blue-green with a manual switch after testing
Moves everyone at once; no gradual real-user signal.

C The Recreate strategy on a Deployment
Everyone at once, with downtime.

D Shadow traffic sent to the new fare engine
Real users never see its answers, so user errors are not measured.

THE SIGNAL

'A few real users, gradually more, automatic rollback': an automated canary.

> Quayside's pipeline builds the booking API image twice from the same commit: once for staging, once for production.
>
> A production bug never appeared in staging. The production build had pulled a newer version of a library.

Which practice prevents this?

A Rebuild more carefully, pinning the OS packages

B Tag both builds latest so they always match

C Build once, then promote that same image

D Skip staging and test only in production

◆ PAUSE AND DECIDE

Which practice prevents this?

A Rebuild more carefully, pinning the OS packages

Helps, but two builds can still differ. One build cannot.

B Tag both builds latest so they always match

A shared tag hides the difference; it does not remove it.

C Build once, then promote that same image

Production runs exactly the bytes staging tested.

D Skip staging and test only in production

Removes the safety net instead of fixing the pipeline.

THE SIGNAL

Two builds from one commit can differ. Build once, promote by digest.

Module 22 in five lines

- **CI** builds and tests an artefact; **CD** moves it into environments.
- Continuous **delivery** has a human gate; continuous **deployment** does not.
- **Build once, promote the same image;** only configuration differs.
- **Blue-green:** everyone at once, instant rollback. **Canary:** a few first, then more.
- **Progressive delivery** automates canaries; **feature flags** separate deploy from release.

Every environment needs slightly different manifests. Next: **Helm and Kustomize**.